1. Tp sur bases de spring-framework

Récupérer une copie de https://github.com/didier-tp/spring2026.git (via ***git clone*** ou via **code/download-zip** + extraction du zip dans c:\tp ou ailleurs)

1.1. (Tp facultatif) Très rapide aperçu sur ancienne config XML

L'ancienne configuration de Spring au format XML est aujourd'hui considérée comme obsolète. Elle est néanmoins toujours supportée par spring6 et spring7.
Ce Tp facultatif n'est à priori utile que si besoin de comprendre la structure d'une ancienne application Spring.

L'application exemple "***oldXmlSpringApp***" (au format "maven") du référentiel git
https://github.com/didier-tp/spring2026.git
est un **exemple simple** de **configuration Spring XML** .

Il est possible de charger ce projet dans un IDE tel que eclipse ou intelliJ pour ensuite lancer l'application ou bien les tests unitaires .
La partie configuration XML se situe dans le sous répertoire **src/main/resources** .

1.2. Chargement du projet et analyses/vérifications

L'objectif de cette première série de Tps est d'appréhender les fonctionnalités essentielles de Spring via une approche très progressive en partant volontairement d'un début de projet uniquement basé sur **spring-framework** (sans spring-boot) .

Charger dans eclipse ou intelliJ le projet suivant (au format maven) :
tp/*debutAppliSpringSansSpringBoot (de https://github.com/didier-tp/spring2026.git)*

NB: ce projet nécessite **java 17** comme version minimum du java .
Avec **intelliJ** , vérifier si besoin les réglages java/jdk via le menu **"file / project structure"**

- Analyser la structure de **pom.xml** (avec packaging="war")
- Repérer les principales dépendances (spring-context , …)
- Lancer l'exécution de **tp.appliSpring.exemple.ExempleApp**
- Analyser le code initial de cet exemple simple
- Beaucoup de choses seront approfondies ultérieurement

1.3. Bases de l'injection de dépendance (avec @Autowired)

- dans package ***tp.appliSpring.exemple*** , Répérer la classe ***Coordinateur*** avec le début de code suivant (à compléter)

```
package tp.appliSpring.exemple;
//...
@Component
```

```
public class Coordinateur {
//…
private MonAfficheur monAfficheur=null; //référence vers afficheur à injecter

//...
private MonCalculateur monCalculateur=null;//référence vers calculateur à injecter

public void calculerEtAfficher() {
                double x=4;
                double  res =monCalculateur.calculer(x); //x*x ou bien 2*x ou bien ...
                monAfficheur.afficher("res="+res);// >> res=16 en v1 ou bien ** res=16
        }
}
```

- Au sein de la méthode main() de la classe ExempleApp , ajouter un bloc de code de ce genre :

```
Coordinateur coordinateurPrisEnChargeParSpring =
                        contextSpring.getBean(Coordinateur.class);
coordinateurPrisEnChargeParSpring.calculerEtAfficher();
```

- **Compléter le code de la classe Coordinateur (et ajuster si besoin d'autres classes) de manière à ce que cet exemple fonctionne bien.**
- On pourra coder et tester successivement plein de variantes d'injection de dépendances :
  - via **@Autowired** (ou bien @Resource ou bien @Inject)
    sans ou avec affichage des éléments injectés au sein du constructeur par défaut de la classe Coordinateur et d'une méthode initialiser() préfixée par **@PostConstruct**
  - via des ajouts de ***MonAfficheurV2*** (avec préfixe "**" plutôt que ">>")
    et ***MonCalculateurDouble*** (2*x plutôt que x*x) de manière à engendrer une ambiguité.
  - Via des ajouts de **@Qualifier** pour lever les ambiguités
  - Via une expérimentation de l'**injection par constructeur** (par exemple dans une classe "CoordinateurAvecInjectionParConstructeur")

1.4. Configurations via classes java (@Configuration, @Bean)

- Analyser la structure de la partie **tp.appliSpring.explicit**
  .*app*.SpringAppWithExplicitConf avec main()
  .beans/ avec interfaces et classes
  sans aucune annotation spring
  .conf.ExempleConfigExplicite à compléter en Tp

A faire en Tp:

phase 0: faire fonctionner l'exemple tel quel

phase 1: tenir compte des choix de prefixe et suffixe de exemples.properties via @Value("${...:..}")
avec dans src/main/resources/exemples.properties

```
preferences.prefixe=>>>
```

```
preferences.suffixe=<<<
```

tester le comportement en modifiant les valeurs des prefixe et suffixe dans le .properties

phase 2: variante ...Basic en l'absence de profile "maj"
et variante ...Maj si présence du profile "maj"
tester le comportement en activant ou pas de profile "maj" en début de main()

NB: cette partie "***explicit***" est plus complexe que la partie "***exemple***" et n'a pas beaucoup d'intérêt tel quel .
Par contre, au sein d'un projet plus complexe, la configuration explicite basée sur @Bean peut s'avérer très utile pour paramétrer des composants "spring" basés sur des classes (récupérées via maven depuis une librairie externe) dont on n'a pas le droit de changer le code source .

1.5. Mise en place d'un aspect de type "log automatique"

Mettre en place un **aspect** (via Spring AOP , paramétré via annotations de AspectJ) qui affichera des lignes de logs pour chaque appel d'une méthode d'une classe du package ***tp.appliSpring.exemple*** **.**
On pourra par exemple préciser le temps d'exécution et les noms des méthodes invoquées.

Code initial à compléter :

```
tp.appliSpring.annotation.LogExecutionTime
tp.appliSpring.aspect.MyPerfLogAspect
```

Phase 1 du Tp :

compléter (via ajout d'annotations et de code) les classes suivantes pour que l'aspect soit déclenché sur l'exécution des méthodes des classes du package tp.appliSpring.exemple lorsque le profile "perf" est activé :

```
tp.appliSpring.aspect.MyPerfLogAspect
tp.appliSpring.exemple.ExempleConfig
tp.appliSpring.exemple.ExempleApp.main()
```

Phase 2 du Tp :

Conditionner le comportement précédent au fait que l'annotation **@LogExecutionTime** doit être placée au dessus des méthodes sur lesquelles on souhaite obtenir un temps d'exécution .